CS 391
Web Application Development

Esma Meral

esma.meral@ozyegin.edu.tr

Week - 4

Outline

JavaScript

1. Functions
2. Arrow Functions
3. Objects & Classes
4. Events
5. Callback Functions
6. Arrays
7. Important Array Methods

Functions

JavaScript

1. Functions
2. Arrow Functions
3. Objects & Classes
4. Events
5. Callback Functions
6. Arrays
7. Important Array Methods

JavaScript Functions…

- Functions are named statements that performs tasks
- JavaScript has many built-in functions, and you can write your own
  - parseFloat(), parseInt(), sin(), cos(), log(), toString(), valueOf(), getFullYear(), getDate(), setDate(), many more…
- Inside the function, the parameters behave as **local variables**

keyword

function name

parameters

```
function doWhatever(a,b,c) {
    statement here
}
```

statements of the function

…JavaScript Functions

- Two syntaxes:

```
function multiply(a, b) {
    return a * b;
}

var add = function(a, b) {
    return a + b
};
```

Parameter Rules

- No specify data types
- No type checking
- No number of arguments checking
  - If a function is called with missing arguments (less than declared), the missing values are set to undefined
- Objects are passed by reference

Inner functions

- Functions do not all have to be defined at the top level
- Functions can be defined inside of other functions.

```
function hypotenuse(a, b) {
   function square(x) { return x*x; }

   return Math.sqrt(square(a) + square(b));
}
```

Calling a Function

```
function sayHi() {
     console.log('Hello World!');
}

function sayHello(name) {
     console.log('Good morning '+name);
}

sayHi();
sayHello("Ali");
sayHello();
```

```
Hello World!
Good morning Ali
Good morning undefined
```

Return Statement

- A JavaScript function can have an optional return statement
- A function with an empty return directive or without it returns ***undefined***.

```
function getRectArea(width, height) {
    if (width > 0 && height > 0) {
        return width * height;
    }
    return 0;
}
```

Function Examples (1/3)

JavaScript

```
var element = document.getElementById("header");
element.innerHTML = "New Header";

document.getElementById("p2").style.color = "blue";
```

Function Examples (2/3)

JavaScript

```
<script>
  function calculateAge() {
      var date = new Date();
      var year = date.getFullYear();
      var birthYear = document.getElementById('birthYear').value;
      var age = year - birthYear;
      document.getElementById('age').value = age;
      return false;
  }
</script>
```

Web page

HTML

```
    <form onsubmit="return calculateAge()" >
        <label for="birthYear">Birth Year</label>
        <input type="number" id="birthYear" min="1960" max="2022" required>
        <input type="number" id="age" min="1" max="99" readonly disabled>
        <input type="submit" value="Calculate My Age">
    </form>
```

Notes About Submitting Forms…

- The submit event fires when the user clicks a **submit** button or **presses Enter** while editing a field (e.g. <input type="text">) in a form.
  - Submit buttons:

```
<form ...>
   <input type="submit" value="Calculate Age">
   <button type="submit">Find My Age</button>
</form>
```

…Notes About Submitting Forms

- <input type="button"> can be used anywhere, not just within form and they do not submit form if they are in one.
- <input type="submit"> should **be used in forms** only and they will send a request (either GET or POST) to specified URL.
- The difference is that <button> can have content, whereas <input> cannot
- If you return *false* from an **onsubmit** event handler, this will prevent the submit of the form to the server from happening

Function Examples (3/3)

- Form Validation

JavaScript

```
<script type="text/javascript">
    function validateForm(){
        var fnm = document.forms["myForm"]["firstname"].value;
        if ( fnm == null || fnm == "" ) {
                alert("First name must be filled out");
            return false;
        }
    }
</script>
```

HTML

```
<form name="myForm" action="index.html" onsubmit="return validateForm()" method="post">
   First name: <input type="text" name="firstname">
   <input type="submit" value="Submit">
</form>
```

Arrow Functions

JavaScript

1. Functions
2. Arrow Functions
3. Objects & Classes
4. Events
5. Callback Functions
6. Arrays
7. Important Array Methods

Arrow Functions

- Arrow functions were introduced in ES6.
- Arrow functions allow us to write shorter function syntax:

Before:

```
hello = function() {
  return "Hello World!";
}
```

With Arrow Function:

```
hello = () => {
  return "Hello World!";
}
```

Arrow Functions Return Value by Default:

```
hello = () => "Hello World!";
```

Arrow Function With Parameters

One param. With simple expression return is not needed:

```
param => expression
```

Multiple params require parentheses. With simple expression return is not needed:

```
(param1, paramN) => expression
```

Multiline statements require body braces and return:

```
param => {
  let a = 1;
  return a + param;
}
```

Multiple params require parentheses. Multiline statements require body braces and return:

```
(param1, paramN) => {
   let a = 1;
   return a + param1 + paramN;
}
```

Arrow Function Example

Arrow functions simplify the creation of functions:

```
function sum(a, b){
    return a + b;
};
```

Becomes

```
const sum=(a, b) => a + b  ;
```

Arrow Function Example

Arrow functions make it easy to create functions:

```
var fullnames = people.filter(function (person) {
                  return person.age >= 18;
                }).map(function (person) {
                  return person.fullname;
                });
```

Becomes

```
var fullnames = people.filter(p => p.age >= 18).map(p => p.fullname);
```

Objects & Classes

JavaScript

1. Functions
2. Arrow Functions
3. Objects & Classes
4. Events
5. Callback Functions
6. Arrays
7. Important Array Methods

Objects

- Objects are variables that can store both values and functions.
- Values are stored as *key:value* pairs called properties

properties

method

```
const student = {
        firstName:"Kerem",
        lastName:"Kara",
        number:"S123456",
        program:"CS",
        fullName : function() {
                return this.firstName + " " + this.lastName;
        }
};
```

Property values

this refers to the object.

Classes

- ES6, introduced JavaScript Classes.
- JavaScript Classes are templates for JavaScript Objects.

```
class Student {
    constructor(firstName, lastName, program) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.program = program;
    }
    fullName() {
        return this.firstName + " " + this.lastName;
    }
}
const myStudent = new Student("Kerem", "Kara", "CS");
console.log(myStudent.fullName());
```

Classes and Inheritance

- ES6 introduces classes and a way to create classical OOP

```
 class SeniorStudent extends Student {
       constructor(firstName, lastName, program, advisor) {
            super(firstName, lastName, program);
            this.advisor = advisor;
       }
       info() {
            return this.fullName() + ', Advisor:' + this.advisor;
       }
}

const myStudent = new SeniorStudent("Kerem", "Kara", "CS", "Hasan");
console.log(myStudent.info());
```

Object Literals

ES6 adds a new feature to the way of defining properties

```
let name = 'Ceren Aksu';
     age = 25;
let person = {
    name,
    age
};
```

Instead of

```
let name = 'Ceren Aksu',
     age = 25;
let person = {
     name: name,
     age: age
};
```

Destructuring Assignments

- Destructuring assignments allow to set values to objects in an easier way:
  - Destructuring assignments with objects:

```
const student = {
  name:   'Berke',
  age:   20,
  city: 'İstanbul'
};

const { name, age, city } = student;
```

Instead of

```
var student = {
  name: 'Berke',
  age: 20,
  city: 'İstanbul'
};
    var name = student.name,
    age = student.age,
    city = student.city;
```

Events

JavaScript

1. Functions
2. Arrow Functions
3. Objects & Classes
4. Events
5. Callback Functions
6. Arrays
7. Important Array Methods

What is an Event?

- JavaScript's interaction with HTML is handled through events
  - When the page loads, it is called an event.
  - When the user clicks a button, that click is an event.
  - Pressing any key, closing a window, resizing a window, etc. are also events
- Events are a part of the Document Object Model (DOM)
- Every HTML element can contain a set of events that can trigger JavaScript Code

How to specify an event handler for a particular event?

- Two ways:
  1. Adding an HTML attribute named *on<eventtype>:*
     - <button onclick="handleClick()">
  2. Setting the corresponding property from JavaScript:
     - document.querySelector("button").onclick = function(event) { … }

Event Example (First way)

HTML

```
<h1 onclick="this.innerHTML='Ooops!'">
      Click on this text!
</h1>
```

Event Example (Second way)

HTML

```
<h3 id="h3">
   Second way for using events
</h3>
<script>
   document.getElementById("h3").onclick = function (event) {
                    this.innerHTML = "Hello"; }
</script>
```

onclick Event Type

- **onclick():** occurs when a user clicks the left button of his mouse.
  - You can put your validation, warning etc., against this event type.

```
<body>
    <p>Click the following button</p>

    <input type="button" onclick="alert('Hello World')" value="OK" />

</body>
```

onsubmit Event Type

- **onsubmit** is an event that occurs when you try to submit a form. You can put your form validation against this event type.

```
<html>
  ...
      <form method = "POST" action = "..." onsubmit = "return validate()">
         .......
         <input type = "submit" value = "Submit" />
      </form>
   ...
</html>
```

```
<script>
function validation() {
   all validation goes here
   .........
  return either true or false
}
</script>
```

onmouseover and onmouseout Event Types

- **onmouseover():** triggers when you bring your mouse over any element
- **onmouseout():** triggers when you move your mouse out from that element.

```
<script>
function mouseOver() {
    document.getElementById("test").style.color = "red";
}

function mouseOut() {
     document.getElementById("test").style.color = "black";
}
</script>
```

```
<div onmouseover="mouseOver()" onmouseout="mouseOut()">
  <h2 id="test"> This is inside the division </h2>
</div>
```

This is inside the division

This is inside the division

For More Info

- https://www.w3schools.com/js/js_functions.asp
- https://www.w3schools.com/Jsref/dom_obj_event.asp

Practice with **js-sample9.html** and **addAnimal.html**

Callback Functions

JavaScript

1. Functions
2. Arrow Functions
3. Objects & Classes
4. Events
5. Callback Functions
6. Arrays
7. Important Array Methods

What is a Callback Function?

- In JavaScript, functions are objects
- Functions can be passed to other functions as parameters

```
function print(callback) {
     callback();
}
```

- The ***print( )*** function takes another function as a parameter and calls it inside.
- So, a function that is passed to another function as a parameter is a callback function.

Why do we need Callback Functions?

- JavaScript runs code sequentially in top-down order.
- However, there are some cases that code must run after something else happens and not sequentially.
- This is called asynchronous programming.
- Callbacks make sure that a function is not going to run before a task is completed but will run right after the task has completed.
  - It helps us develop asynchronous JavaScript code and keeps us safe from problems and errors.

Callback Example-1

```
var message = function () {
    console.log("This message is shown after 3 seconds");
}
console.log("Hello");
setTimeout(message, 3000);
console.log("Bye bye");
```

message function is being called after something happened (after 3 seconds passed for this example)

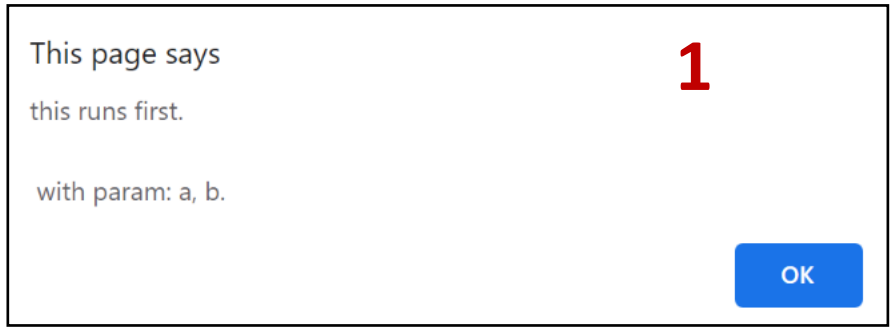
```
Hello
Bye bye
This message is shown after 3 seconds
```

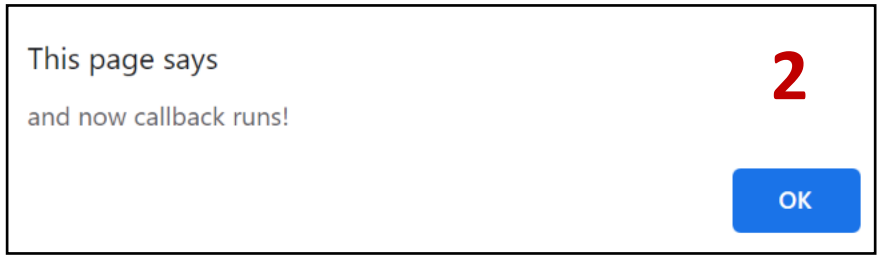
"setTimeout" is a built-in method in JavaScript
Calls a function or evaluates an expression after a given period of time (in milliseconds)

Callback Example-2

```
function myFunction(param1, param2, callback) {
    alert('this runs first.\n\n with param: ' + param1 + ', ' + param2 + '.');
    callback();
}

myFunction('a', 'b', function () {
      alert('and now callback runs!');
});
```





What is an Anonymous Function?

- A function can be defined directly inside another function

```
<script>
    console.log("Hello");
    setTimeout(function () {
        console.log("This message is shown after 3 seconds");
    }, 3000);
   console.log("Bye bye");
</script>
```

Anonymous Function

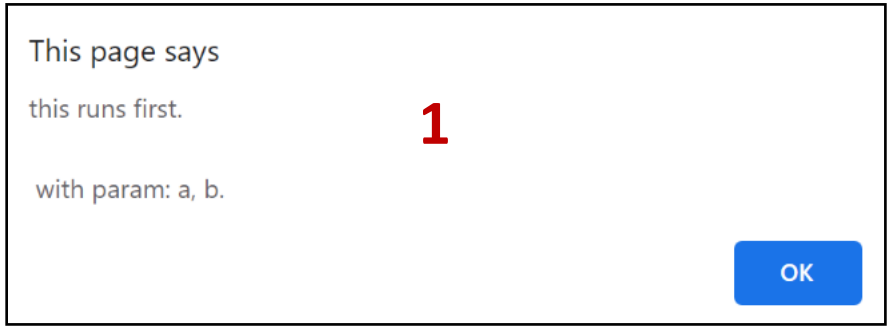
Callback Example-3…

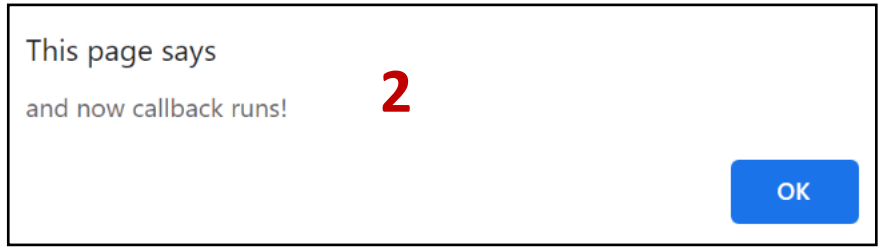
```
function myFunction(param1, param2, callback) {
     alert('this runs first.\n\n with param:' +param1+', '+ param2+'.');
     setTimeout(function () { alert("Hi") }, 5000);
     callback();
}

myFunction('a', 'b', function () {
     alert('and now callback runs!');
});
```

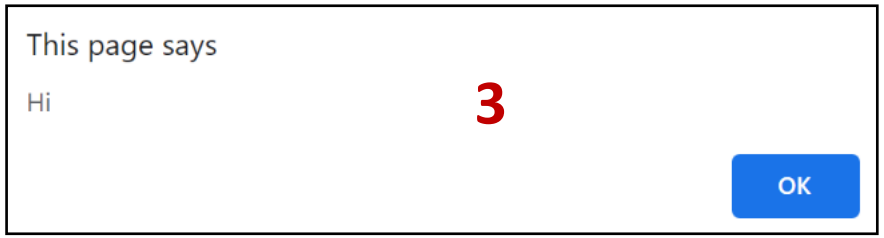
alert('Hi') will run AFTER the callback.

...Callback Example-3





After 5 seconds:



Practice with **js-sample10.html**

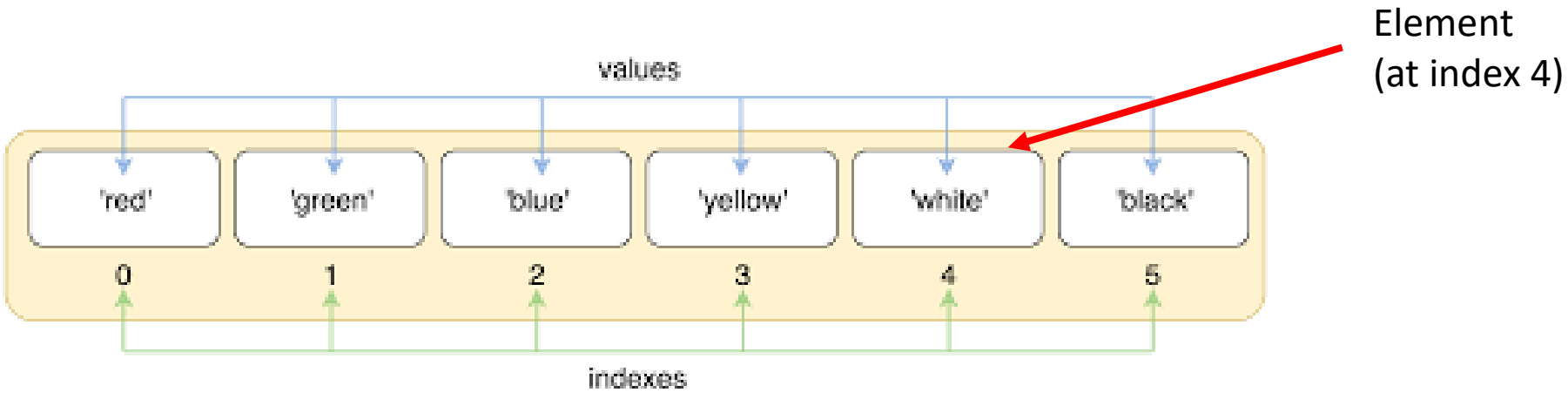
Arrays

JavaScript

1. Functions
2. Arrow Functions
3. Objects & Classes
4. Events
5. Callback Functions
6. Arrays
7. Important Array Methods

JS Arrays

- Arrays are used for storing multiple values in a single variable
- Stores sequential collection of elements
- Access values by referring to an index number



Creating Arrays

- Two syntaxes for generating arrays
- Second syntax is used most of the time.

```
var names = new Array("ali", "kerem", "eda");

var names = ["ali", "kerem", "eda"];
```

Creating Arrays

this syntax is not used frequently:

```
var pets=["Cat", "Dog", "Bird"];
```

```
var pets=new Array("Cat", "Dog", "Bird");
```

```
var pets = ["Cat", "Dog", "Bird"];
console.log(pets[0]); // Cat
console.log(pets[1]); // Dog
console.log(pets[2]); // Bird
pets[3]="Rabbit";
console.log(pets[3]); // Rabbit
pets[2]="Hamster"; // now ["Cat", "Dog", "Hamster", "Rabbit"]
console.log(pets[2]); // Hamster
console.log(pets);  // ["Cat", "Dog","Hamster","Rabbit"]
console.log(pets.length); // 4
```

```
Cat
Dog
Bird
Rabbit
Hamster
▶ (4) ["Cat", "Dog", "Hamster", "Rabbit"]
4
```

Practice with **js-sample7.html**

Looping Array Elements (1/4)

JS

```
<script>
    var pets = ["Cat", "Dog", "Bird", "Rabbit", "Hamster"];
    var text = "<ul>";
    for (i = 0; i < pets.length; i++) {
        text += "<li>" + pets[i] + "</li>";
    }
    text += "</ul>";

    document.getElementById("pets").innerHTML = text;
</script>
```

RESULT

Pet List

- Cat
- Dog
- Bird
- Rabbit
- Hamster

HTML

```
<div id="pets"></div>
```

Looping Array Elements (2/4)

JS Array Definition:

```
<script>
var owners = [
      { firstName: "Linda", lastName: "James", age: 46 },
      { firstName: "John", lastName: "Smith", age: 21 },
      { firstName: "Karen", lastName: "Robert", age: 25 }
    ];
...
</script>
```

Looping Array Elements (3/4)

JS

```
var table = "<table border='1'>";
for (i = 0; i < owners.length; i++) {
  table += "<tr>"
  table += "<td>" + owners[i].firstName + "</td><td> " +
     owners[i].lastName + "</td><td> " +
     owners[i].age + "</td>";
  table += "</tr>"
}
table += "</table>";

document.getElementById("owners").innerHTML = table;
```

RESULT

Pet Owners

Linda	James	46
John	Smith	21
Karen	Robert	25

HTML

```
<div id="owners"></div>
```

Looping Array Elements (4/4)

JS

```
var tableText = "<table border='1'>";
    for (owner of owners) {
        tableText  += "<tr>"
        tableText  += "<td>" + owner.firstName + "</td><td> " +
        owner.lastName + "</td><td> " +
        owner.age + "</td>";
        tableText  += "</tr>"
    }
tableText  += "</table>";

document.getElementById("owners").innerHTML = tableText;
```

RESULT

Pet Owners

Linda	James	46
John	Smith	21
Karen	Robert	25

HTML

```
<div id="owners"></div>
```

Practice with **js-sample8.html**

Important Array Methods

JavaScript

1. Functions
2. Arrow Functions
3. Objects & Classes
4. Events
5. Callback Functions
6. Arrays
7. Important Array Methods

map()

- **<u>Creates a new array</u>** with the results of calling a function for every array element.
- Does not change the original array.
- Returns the created array.
- Syntax:
  - array.map(function(currentValue, index, arr), thisValue)

required **optional** **optional**

Map Examples…

```
var numbers = [25, 121, 9, 81];
var roots = numbers.map(Math.sqrt);
document.write("roots are : " + roots);
```

roots are : 5,11,3,9

Practice with: js-sample11.html

...Map Examples

```
var arr = [6.5, 21, 3, 5, 7];
var nums = arr.map(myFunction)

function myFunction(num) {
   return num * 10;
}
```

nums : 65,210,30,50,70

Practice with: js-sample11.html

sort()…

- Sorts the items of an array.
  - **<u>This method is used for sorting the array in place, changing the element order.</u>**
- Sort order can be either alphabetic or numeric, and either ascending or descending.
- By default, the sort() method sorts the values as strings in alphabetical and ascending order.

```
var names = ["Naci","Zeynep","Hasan","Ahmet"];
console.log(names);
console.log(names.sort());
```

```
["Naci", "Zeynep", "Hasan", "Ahmet"]
["Ahmet", "Hasan", "Naci", "Zeynep"]
```

...sort()

Practice with: js-sample11.html

- Defining alternative sort order:
  - array.sort(compareFunction)
  - function should return a negative, zero, or positive value, depending on the arguments, like:
    - function(a, b){return a-b}
  - If the result is negative **a is sorted before b** otherwise b is sorted before a.

```
var numbers = [40, 100, 1, 5, 25, 10];
console.log(numbers.sort());
```

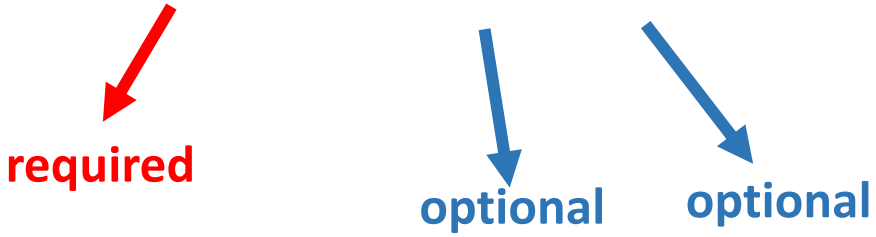
```
[1, 10, 100, 25, 40, 5]
```

```
var numbers = [40, 100, 1, 5, 25, 10];
console.log(numbers.sort(function(a, b){
        return a - b})
);
```

```
[1, 5, 10, 25, 40, 100]
```

filter()

- **<u>Creates an array</u>** filled with all array elements that pass a test
- Does not change the original array.
- Syntax:
  - array.filter(function(currentValue, index, arr), thisValue)



Filter Example

```
var ages = [9, 32, 33, 12, 40];
console.log(ages);
function checkAdult(age) {
   return age >= 16;
}
var adults=ages.filter(checkAdult);
console.log(adults);
console.log(ages);
```

```
[9, 32, 33, 12, 40]
[32, 33, 40]
[9, 32, 33, 12, 40]
```

Practice with: js-sample11.html

reduce()

- Reduces the array to a single value.
- Executes a provided function for each value of the array
- The return value of the function is stored in an accumulator (result/total).
- <u>Does not change the original array</u>.
- Syntax:
  - reduce(function(previousValue, currentValue, currentIndex, array) { …}, initialValue)

required required optional optional optional

Reduce Example

```
var numberList = [100, 200, 50];

function sumofArray(prevValue, num)
    return prevValue + num;
}
var result1 = numberList.reduce(sumofArray);
console.log("Reduce result:" + result1);

var result2 = numberList.reduce(sumofArray, 5000);
console.log("Reduce result:" + result2);
```

```
Reduce result:350
```

result=100+200+50 = 350

```
Reduce result:5350
```

result=5000+100+200+50 = 5350

Practice with: js-sample11.html

forEach()

- Executes a provided function for each value of the array
- Syntax:
  - array.forEach(function(currentValue, index, arr), thisValue)

required

optional optional optional

ForEach Example

Standard for loop statement

```
let ranks = ['A', 'B', 'C'];
for (let i = 0; i < ranks.length; i++) {
        console.log(ranks[i]);
}
```

forEach() usage

```
ranks.forEach(function (e) {
    console.log(e);
});
```

Same

```
function printRanks(e) {
    console.log(e);
}
ranks.forEach(printRanks);
```

Practice with: js-sample11.html

ForEach Example

```
var pets = ["Cat", "Dog", "Bird", "Rabbit", "Hamster"];
var text = "<ul>";
for (i = 0; i < pets.length; i++) {
       text += "<li>" + pets[i] + "</li>";
}
text += "</ul>";
document.getElementById("pets").innerHTML = text;
```

Instead of

```
var pets = ["Cat", "Dog", "Bird", "Rabbit", "Hamster"];
var text = "<ul>";
pets.forEach((pet) => text += "<li>" + pet + "</li>");
text += "</ul>";
document.getElementById("pets").innerHTML = text;
```

pop()

- **pop()** - removes the last element from an array and returns that element.

```
var names = ["ali", "hasan", "ceyda"];
console.log(names); // ali, hasan, ceyda
console.log(names.pop()); // remove "ceyda" and print it
console.log(names); // ali, hasan
```

```
▶ (3) ["ali", "hasan", "ceyda"]
ceyda
▶ (2) ["ali", "hasan"]
```

Practice with: js-sample12.html

shift()

- **shift():** removes the first element from an array and returns that element.

```
var names = ["ali", "hasan", "ceyda"];
console.log(names);// ali, hasan, ceyda
console.log(names.shift());//remove ali and print it
console.log(names);// hasan, ceyda
```

```
▶ (3) ["ali", "hasan", "ceyda"]
ali
▶ (2) ["hasan", "ceyda"]
```

Practice with: js-sample12.html

unshift()

- **unshift():** Adds one or more elements to the front of an array and returns the new length of the array.

```
var names = ["ali", "hasan", "ceyda"];
console.log(names);// ali, hasan, ceyda
console.log(names.unshift("demet","kerem")); //5
console.log(names);// demet, kerem, ali, hasan, ceyda
```

```
▶ (3) ["ali", "hasan", "ceyda"]
5
▶ (5) ["demet", "kerem", "ali", "hasan", "ceyda"]
```

Practice with: js-sample12.html

push()

- **push() :** adds one or more elements to the end of an array and returns the new length of the array.

```
var names = ["ali", "hasan", "ceyda"];
console.log(names);// ali, hasan, ceyda
console.log(names.push("demet","kerem")); //5
console.log(names);// ali, hasan, ceyda, demet, kerem
```

```
▶ (3) ["ali", "hasan", "ceyda"]
5
▶ (5) ["ali", "hasan", "ceyda", "demet", "kerem"]
```

Practice with: js-sample12.html

reverse()

- **reverse() :** reverses the order of the elements of an array
  - the first becomes the last, and the last becomes the first..
  - overwrites the original array.

```
var names = ["ali", "hasan", "ceyda"];
console.log(names);// ali, hasan, ceyda
console.log(names.reverse()); //ceyda, hasan, ali
```

```
▶ (3) ["ali", "hasan", "ceyda"]
▶ (3) ["ceyda", "hasan", "ali"]
```

Practice with: js-sample12.html

indexOf()

- **indexOf() :** returns the first (least) index of an element within the array equal to the specified value, or -1 if none is found.

```
var names = ["ali", "hasan", "ceyda"];
console.log(names.indexOf("hasan"));// 1
```

Practice with: js-sample12.html

Question

- What will be printed on the screen when the following program is executed?

```
var names = ["ali", "hasan", "ceyda"];
console.log(names);
names[3]="esma";
names[5]="meral";
console.log(names);
console.log(names.length);
```

Answer

```
Array(6)
0: "ali"
1: "hasan"
2: "ceyda"
3: "esma"
5: "meral"
length: 6
```

```
▶ (3) ["ali", "hasan", "ceyda"]
▶ (6) ["ali", "hasan", "ceyda", "esma", empty, "meral"]
6
```

For More Info

- Visit following web sites:
  - https://www.w3schools.com/jsref/jsref_obj_array.asp
  - https://www.tutorialspoint.com/javascript/javascript_arrays_object.htm
  - https://www.w3docs.com/learn-javascript/array-methods.html
  - https://www.w3schools.com/js/js_array_methods.asp
  - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

Any Question?

Thank you for your attention

esma.meral@ozyegin.edu.tr